Assignment 3

Minnie & Fran

1.1 Introduction, environment, and reward

Task I and definition of actions

Objective: control the car in CarRacingObstacles-v3 with visual state (96×96 RGB) using DeepRL.

Success requirement: average > 500 over the last 20 episodes (in both discrete and continuous modes).

What we compare: DQN vs. Double Dueling DQN, and discrete control vs. discretized continuous control.

Observation: RGB image 96×96 (+ FrameStack)

Actions:

- **Discrete:** 5 actions (nothing/left/right/gas/brake).
- **"Continuous" discretization:** set of 31 actions with steer/gas/brake + recovery.
- **Reward**: -0.1 per frame, + for new tiles, penalty for going off course, collision ends episode.

Methods:

- DQN: policy net + target net, replay buffer, target update every FreezeCounter.
- Double DQN: reduce overestimation (action chosen by policy, evaluated by target).
- Dueling: branches V(s) and A(s,a) and combination with mean for identifiability.

Steering: 5 values {−1.0,−0.5,0.0,0.5,1.0}, covering sharp turns, gentle turns, and straight driving.

Acceleration: 4 values {0.0,0.3,0.6,1.0}, from no acceleration to maximum acceleration.

Brake: 2 values {0.2,0.5}, "real" braking (brake 0 is avoided so as not to duplicate actions).

Two main groups were constructed:

- **Actions with gas (without brake):** combinations [steer, gas, 0.0].
- **Actions with brake (only if gas=0):** combinations [steer, 0.0, brake], so that the agent either accelerates or brakes, but not both at the same time.

In addition, an extra recovery/emergency action was added:

Hard braking straight ahead [0.0,0.0,0.8], useful for avoiding collisions or stabilizing the car in critical situations.

Motivation: "gas and brake simultaneously" is avoided because it tends to generate inconsistent behavior and more noise during learning, hindering policy convergence.

1.2 Architecture used

CNN (Nature-DQN) architecture for estimating Q-values

Classic architecture by Mnih et al. (Nature, 2015), used as a baseline for DQN and variants (Double/Dueling)

https://medium.com/nerd-for-tech/reinforcement-learning-deep-q-learning-with-atari-games-63f5242440b1

Backbone CNN (Nature-DQN)

Step	Layer	Main parameters	Output shape
1	Conv2d	in=C, out=32, kernel=8×8, stride=4, pad=0	(B, 32, 23, 23)
2	ReLU	inplace=True	(B, 32, 23, 23)
3	Conv2d	in=32, out=64, kernel=4×4, stride=2, pad=0	(B, 64, 10, 10)
4	ReLU	inplace=True	(B, 64, 10, 10)
5	Conv2d	in=64, out=64, kernel=3×3, stride=1, pad=0	(B, 64, 8, 8)
6	ReLU	inplace=True	(B, 64, 8, 8)
7	Flatten	64×8×8 → **4096**	(B, 4096)

n_flat is calculated dynamically with a dummy input to avoid hardcoding 4096 if you change convs or channels per FrameStack.

Head DQN (output Q-values)

Step	Layer	Main parameters	Output shape
8	Linear	in=4096, out=512	(B, 512)
9	ReLU	inplace=True	(B, 512)
10	Linear	in=512, out=|A|	(B, |A|)

|A| (number of actions):

- Discrete: 5
- Discretized continuous: size of your action_space = 31

Head Dueling (use_dueling=True)

A) Shared layer

Step	Layer	Main parameters	Output shape
8’	Linear (shared)	in=4096, out=512	(B, 512)
9’	ReLU	inplace=True	(B, 512)

1.3 Architecture used and hyperparameters

Head Dueling (use_dueling=True)

B) Two "heads"

Step	Layer	Main parameters	Output shape
10a	Advantage head (Linear)	in=512, out=|A|	(B, |A|)
10b	Value head (Linear)	in=512, out=1	(B, 1)

$$Q(s,a) = V(s) + A(s,a) - \text{mean}(A(s, \cdot))$$

Practical detail:

Input normalization: before the convs, convert to float and, if it comes in [0..255], divide by 255 to work in [0,1]

Hyperparameters held constant across all 5 models

- MaxSteps = 1000
- LearningRate = 1e-4 (Adam)
- gamma = 0.99
- epsilon_start = 1.0
- TRAIN_EVERY = 4

Category	Hyperparameter	M1 (DQN disc.)	M2 (DQN cont.)	M3 (DD+Duel disc.)	M4 (DD+Duel cont.)	M5 (DD+Duel cont. + Rand)	Comment (EN)
Env / Domain	**UseRandomize**	False	False	False	False	True	Randomization enforces generalization (harder training).
Training	**TotalEpisodes**	500	500	500	600	700	More difficulty typically requires more episodes.
Training	**ControlSteps**	3	2	2	2	2	Fewer repeats make control more reactive/fine-grained.
Training	**FrameStack**	2	4	4	4	4	4-frame stacks capture dynamics, at higher memory/compute cost.
Target	**FreezeCounter**	10	10	10	10	20	Slower target updates can stabilize learning under domain randomization
Optim	**BatchSize**	32	32	32	64	32	Larger batches reduce gradient variance but increase update cost
Early stop	**SOLVE_THRESHOLD**	500	500	600	600	600	Strciter stopping criterion from M3 onward

1.4 Hyperparameters, results + comparisons

Hyperparameters + model 1: DQN + discrete control

Category	Hyperparameter	M1 (DQN disc.)	M2 (DQN cont.)	M3 (DD+Duel disc.)	M4 (DD+Duel cont.)	M5 (DD+Duel cont. + Rand)	Comment (EN)
Replay	**BufferSize**	15k	30k	50k	50k	60k	Randomization benefits from higher replay diversity (larger buffer)
Replay	**MIN_REPLAY_SIZE**	5k	10k	5k	10k	15k	A larger warm-up improves stability before training starts
Exploration	**epsilon_min**	0.05	0.07	0.05	0.07	0.05	With more actions, keeping slightly more exploration can help
Exploration	**epsilon_decay**	0.995	0.996	0.995	0.996	0.997	4-frame stacks capture dynamics, at higher memory/compute cost.
Heuristic	**InitCounter**	30	50	50	50	50	Early termination saves time on clearly failed episodes

Model 1



=== Evaluation: UseRandomize=False ===
Seed 1 | Total Reward: **825.145**
Seed 2 | Total Reward: **641.101**
Seed 3 | Total Reward: **902.499**
Seed 4 | Total Reward: **881.218**
Seed 5 | Total Reward: **793.290**

=== Evaluation: UseRandomize=True ===
Seed 6 | Total Reward: **659.910**
Seed 7 | Total Reward: **127.613**
Seed 8 | Total Reward: **223.549**
Seed 9 | Total Reward: **758.742** ✅
Seed 10 | Total Reward: **250.199**

--- **No randomize (seeds 1-5)** ---
Mean: **808.651** | Std: **92.363** | Min: **641.101**
Disasters (reward < 200): **0/5**

--- **Randomize (seeds 6-10)** ---
Mean: **404.003** | Std: **254.560** | Min: **127.613**
Disasters (reward < 200): **1/5**

--- **ALL (seeds 1-10)** ---
Mean: **606.327** | Std: **278.560** | Min: **127.613**
Disasters (reward < 200): **1/10**

1.5 Results + comparisons

Model 2: DQN + continuous control
Model 3: Double Dueling DQN + discrete control

Model 2



=== **Evaluation: UseRandomize=False** ===
Seed 1 | Total Reward: **1006.191**
Seed 2 | Total Reward: **433.178**
Seed 3 | Total Reward: **871.679**
Seed 4 | Total Reward: **776.142**
Seed 5 | Total Reward: **781.192**

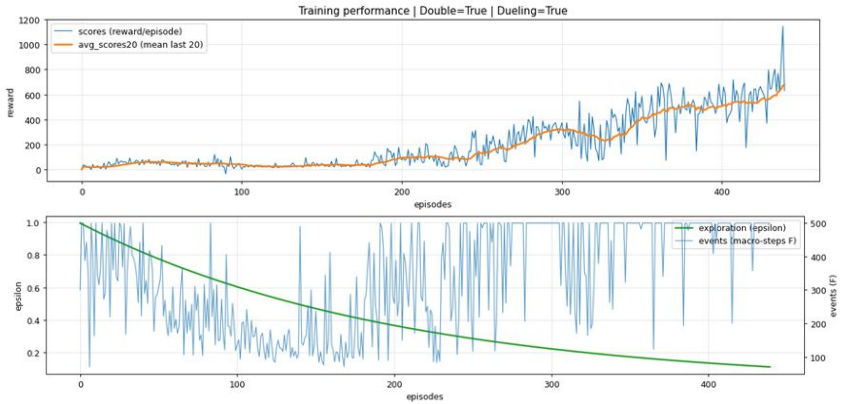
=== **Evaluation: UseRandomize=True** ===
Seed 6 | Total Reward: **-11.600**
Seed 7 | Total Reward: **-42.391**
Seed 8 | Total Reward: **-63.626**
Seed 9 | Total Reward: **514.648**
Seed 10 | Total Reward: **-62.841**

--- **No randomize (seeds 1-5)** ---
Mean: **773.676** | Std: **189.562** | Min: **433.178**
Disasters (reward < 200): **0/5**

--- **Randomize (seeds 6-10)** ---
Mean: **66.838** | Std: **224.702** | Min: **-63.626**
Disasters (reward < 200): **4/5**

--- **ALL (seeds 1-10)** ---
Mean: **420.257** | Std: **410.021** | Min: **-63.626**
Disasters (reward < 200): **4/10**

Model 3




```
=== Evaluation: UseRandomize=False ===
Seed  1 | Total Reward: 952.745
Seed  2 | Total Reward: 483.118
Seed  3 | Total Reward: 956.270
Seed  4 | Total Reward: 616.609
Seed  5 | Total Reward: 565.684

=== Evaluation: UseRandomize=True ===
Seed  6 | Total Reward: 633.726
Seed  7 | Total Reward: -47.600
Seed  8 | Total Reward: -59.435
Seed  9 | Total Reward: -28.520
Seed 10 | Total Reward: -70.841
```

```
--- No randomize (seeds 1-5) ---
Mean: 714.885 | Std: 200.239 | Min: 483.118
Disasters (reward < 200): 0/5

--- Randomize (seeds 6-10) ---
Mean: 85.466 | Std: 274.487 | Min: -70.841
Disasters (reward < 200): 4/5

--- ALL (seeds 1-10) ---
Mean: 400.176 | Std: 395.931 | Min: -70.841
Disasters (reward < 200): 4/10
```

1.6 Results + comparisons

Model 4: Double Dueling DQN + continuous control
Model 4 Randomize

Model 4




```
=== Evaluation: UseRandomize=False ===
Seed  1 | Total Reward: 800.255
Seed  2 | Total Reward: 537.790
Seed  3 | Total Reward: 813.608
Seed  4 | Total Reward: 680.660
Seed  5 | Total Reward: 533.447

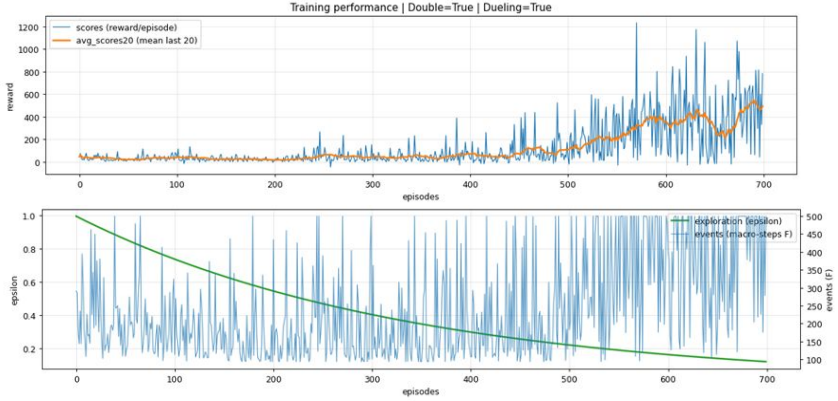
=== Evaluation: UseRandomize=True ===
Seed  6 | Total Reward: -55.901
Seed  7 | Total Reward: 10.649
Seed  8 | Total Reward: -60.248
Seed  9 | Total Reward: 460.432
Seed 10 | Total Reward: 82.374
```

```
--- No randomize (seeds 1-5) ---
Mean: 673.152 | Std: 121.474 | Min: 533.447
Disasters (reward < 200): 0/5

--- Randomize (seeds 6-10) ---
Mean: 87.461 | Std: 193.599 | Min: -60.248
Disasters (reward < 200): 4/5

--- ALL (seeds 1-10) ---
Mean: 380.306 | Std: 334.480 | Min: -60.248
Disasters (reward < 200): 4/10
```

Model 4 Randomize




```
=== Evaluation: UseRandomize=False ===
Seed  1 | Total Reward: 1155.136
Seed  2 | Total Reward: 534.810
Seed  3 | Total Reward: 1172.102
Seed  4 | Total Reward: 1281.936
Seed  5 | Total Reward: 921.770

=== Evaluation: UseRandomize=True ===
Seed  6 | Total Reward: -54.932
Seed  7 | Total Reward: 405.457
Seed  8 | Total Reward: 343.115
Seed  9 | Total Reward: 1201.350
Seed 10 | Total Reward: -39.294
```

```
--- No randomize (seeds 1-5) ---
Mean: 1013.151 | Std: 266.395 | Min: 534.810
Disasters (reward < 200): 0/5

--- Randomize (seeds 6-10) ---
Mean: 371.139 | Std: 456.334 | Min: -54.932
Disasters (reward < 200): 2/5

--- ALL (seeds 1-10) ---
Mean: 692.145 | Std: 492.594 | Min: -54.932
Disasters (reward < 200): 2/10
```

1.7 Results + comparisons

Model 4 Randomize with obstacles

Model 4 with obstacles

```
=== Evaluation: UseRandomize=False ===
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 54
[CarRacingObstacles] Placed mountain around tile 109
[CarRacingObstacles] Placed mountain around tile 164
[CarRacingObstacles] Placed mountain around tile 219
Seed  1 | Total Reward: 485.698
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 66
[CarRacingObstacles] Placed mountain around tile 133
[CarRacingObstacles] Placed mountain around tile 200
[CarRacingObstacles] Placed mountain around tile 267
Seed  2 | Total Reward: 862.084
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 54
[CarRacingObstacles] Placed mountain around tile 108
[CarRacingObstacles] Placed mountain around tile 162
[CarRacingObstacles] Placed mountain around tile 216
Seed  3 | Total Reward: 472.636
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 54
[CarRacingObstacles] Placed mountain around tile 109
[CarRacingObstacles] Placed mountain around tile 164
[CarRacingObstacles] Placed mountain around tile 219
Seed  4 | Total Reward: 808.436
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 65
[CarRacingObstacles] Placed mountain around tile 131
[CarRacingObstacles] Placed mountain around tile 196
[CarRacingObstacles] Placed mountain around tile 262
Seed  5 | Total Reward: 726.960
```

```
=== Evaluation: UseRandomize=True ===
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 62
[CarRacingObstacles] Placed mountain around tile 124
[CarRacingObstacles] Placed mountain around tile 186
[CarRacingObstacles] Placed mountain around tile 248
Seed  6 | Total Reward: -31.408
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 53
[CarRacingObstacles] Placed mountain around tile 107
[CarRacingObstacles] Placed mountain around tile 160
[CarRacingObstacles] Placed mountain around tile 214
Seed  7 | Total Reward: 11.314
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 59
[CarRacingObstacles] Placed mountain around tile 118
[CarRacingObstacles] Placed mountain around tile 177
[CarRacingObstacles] Placed mountain around tile 236
Seed  8 | Total Reward: -39.978
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 61
[CarRacingObstacles] Placed mountain around tile 122
[CarRacingObstacles] Placed mountain around tile 183
[CarRacingObstacles] Placed mountain around tile 244
Seed  9 | Total Reward: 1112.910
[CarRacingObstacles] Placed mountain around tile 0
[CarRacingObstacles] Placed mountain around tile 54
[CarRacingObstacles] Placed mountain around tile 109
[CarRacingObstacles] Placed mountain around tile 163
[CarRacingObstacles] Placed mountain around tile 218
Seed 10 | Total Reward: 542.147

--- No randomize (seeds 1-5) ---
Mean: 671.163 | Std: 162.615 | Min: 472.636
Disasters (reward < 200): 0/5

--- Randomize (seeds 6-10) ---
Mean: 318.997 | Std: 453.081 | Min: -39.978
Disasters (reward < 200): 3/5

--- ALL (seeds 1-10) ---
Mean: 495.080 | Std: 383.234 | Min: -39.978
Disasters (reward < 200): 3/10
```

- Model 4 is strong and stable in conditions close to those of training (UseRandomize=False), even with obstacles.

- However, it suffers from a clear global generalization problem (UseRandomize=True): although obstacles sometimes lead to excellent episodes, it does not do so reliably.

- This justifies retraining with UseRandomize=True to learn a policy that is more invariant to changes in the environment.

1.8 Results + comparisons

FINAL COMPARISON

1) Model 1: DQN + discrete

- **Evolution:** learns, but more slowly and with a lot of variance (very good episodes mixed with bad ones). It ends up reaching avg20 ≈ 500–520 → meets the objective, but with instability.
- **Reading:** discrete control (5 actions) limits the finesse of the handling, which is why it is difficult to stabilize.

2) Model 2: DQN + continuous (discretized)

- **Evolution:** remains at a plateau for a long time and then takes off strongly (avg20 exceeds 500 earlier, even with early stop). Can achieve "perfect" episodes (>900–1000), but has catastrophic seeds in evaluation.
- **Reading:** discretized continuous control provides more capacity, but basic DQN remains more sensitive/unstable in some scenarios.

3) Model 3: Double Dueling DQN + discrete

- **Evolution:** clear improvement over Model 1: more stable and better final performance (avg20 reaches ~678).
- **Evaluation:** very consistent in UseRandomize=False, but falls sharply in UseRandomize=True (many "disasters").
- **Reading:** Double+Dueling improves learning in the domain, but does not fix generalization if you do not train with randomization.

4) Model 4: Double Dueling DQN + continuous

- **Evolution**: it is the best "without randomization": it learns solidly (avg20 ~610) and is stable in UseRandomize=False.
- **Evaluation:** excellent without randomization, but generalizes poorly with UseRandomize=True (many disasters).
- **Reading:** it is the best "architecture + control" combination, but the bottleneck remains the distribution change (randomize).

5) Randomized Model 4 (retrained with UseRandomize=True)

- **Evolution:** real learning with greater variability (normal), but sustained positive trend.
- **Evaluation:** maintains high performance in UseRandomize=False and improves greatly in UseRandomize=True (no longer crashes as before; reduces disasters).
- **Reading:** it is the best overall model because it balances performance and generalization.

1.9 Code details and conclusions

1) Discretization of continuous control (your big technical decision)

- You went from 5 actions to ~31 to have fine control without exploiting space: steer×gas + steer×brake (only with gas=0) + recovery.
- Justification: DQN needs discrete actions; more actions = better control, but requires slower epsilon decay and more warm-up.

2) FrameStack and ControlSteps

- FrameStack=4 and ControlSteps=2 (continuous) help capture visual dynamics and smooth control.

3) Efficient Replay Buffer (key in Colab)

- Saving states as np.uint8 on CPU and converting to float/255 only when training avoids impossible VRAM/RAM with stacked images.
- Saving simple types (int/float/bool) in replay speeds up sampling and avoids overhead.

4) Warm-up + train every N steps (TRAIN_EVERY)

- Don't train at every step: it reduces cost and improves stability.
- MIN_REPLAY_SIZE avoids starting with a poor and correlated buffer.

5) Checkpoint by BestAvg20 + early stopping

- Saving by best moving average (not by single episode) aligns with the evaluation criteria.
- Early stop when you exceed the threshold saves time and avoids degradation due to noise/exploration.

6) Fix the environment with obstacles

- **Problem:** Box2D fails if it receives np.float32/arrays as position.
- **Solution:** explicit casting to float in positions (and store center as a tuple of floats).

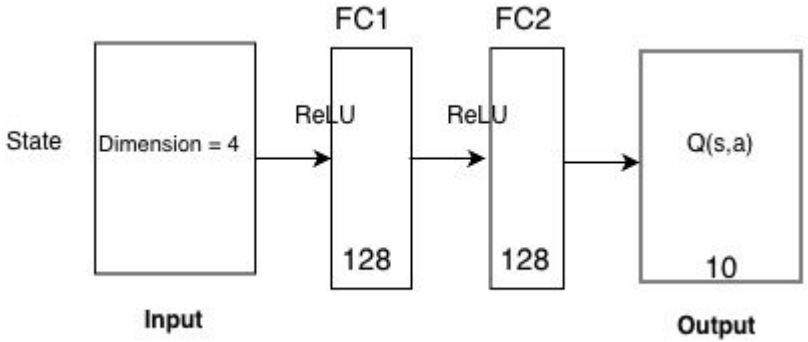
Potential conclusions

- Double + Dueling improves stability and performance within the training domain (seen in higher avg20 and less dependence on “lucky episodes”).
- Training with UseRandomize=False produces visual overfitting (very good performance on similar seeds, but crashes when randomizing).
- Retraining the best model with UseRandomize=True improves generalization (although it increases variance, it reduces “disasters” and raises the average on randomized seeds).

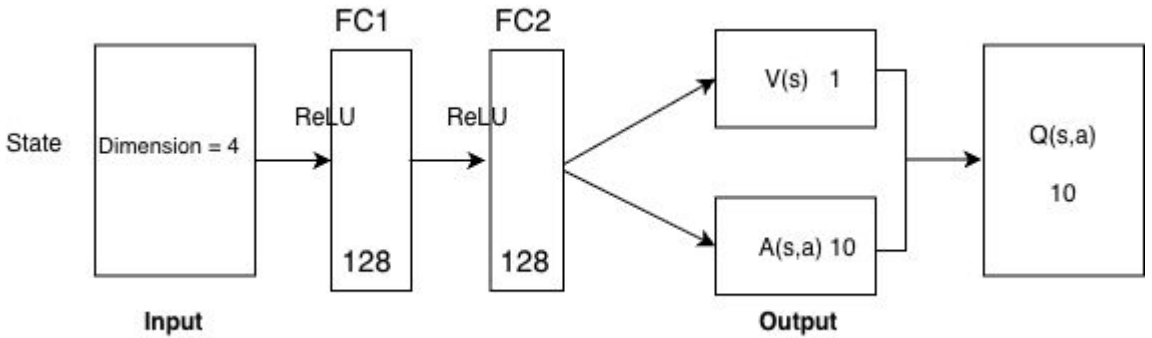
2.1 DQN and Double Dueling DQN Architecture

We implement and compare two reinforcement learning algorithms: **DQN** and **Double Dueling DQN**.

DQN



Double Dueling DQN



2.2 Environment and Hyperparameters

In Part II, No modifications were made to the environment's source code.All experiments were conducted using the original IndoorRobot2025 environment.

We used DIRECTION_AND_ERROR observation mode. This observation mode returns a 4-dimensional state vector, composed of: the normalized direction to the goal (2 values), the robot orientation, the signed lateral distance to the reference path (tracking error).

All agents were trained for 50,000 environment steps using the Adam optimizer.Experience replay and a target network were employed to stabilize training.An ε-greedy exploration strategy was used, with ε linearly decayed from 1.0 to 0.05.

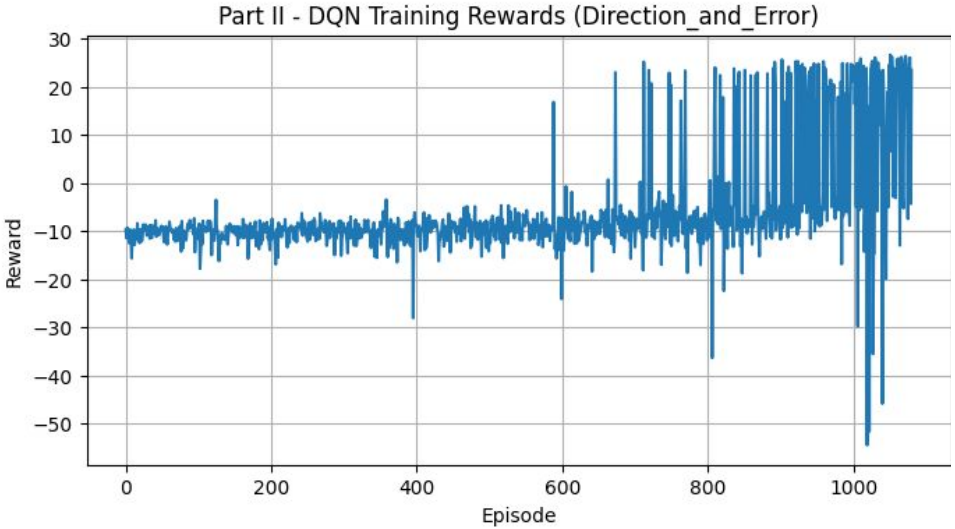
Parameter	**Value**
Discount factor γ	0.99
Learning rate	1e-4
Batch size	128
Replay buffer capacity	50,000
Warm-up steps	2,000
Target network update	Every 1,000 steps
Exploration strategy	ε-greedy
ε range	1.0 → 0.05 (linear decay)

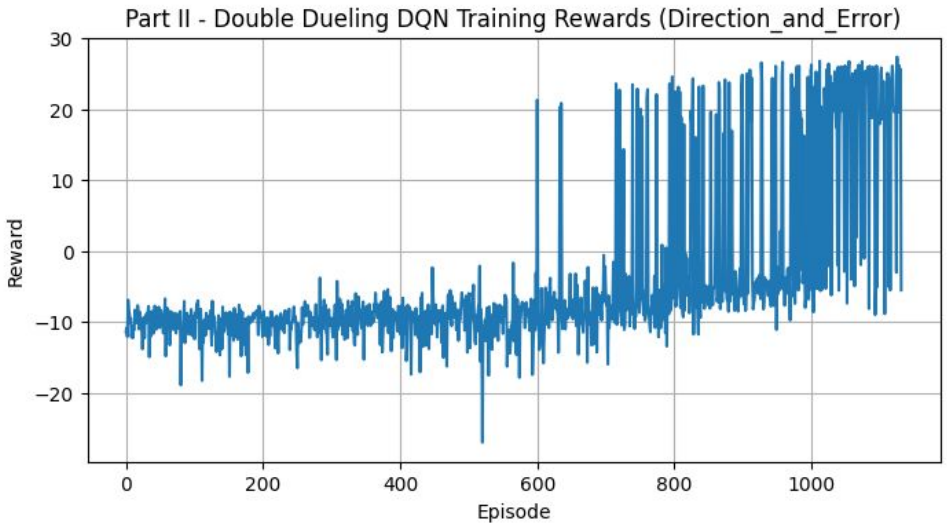
2.3 Results: Training Performance

The **DQN** training curve shows that the agent initially receives predominantly negative rewards, indicating inefficient exploration and poor path tracking. As training progresses, positive reward peaks begin to appear, showing that the agent occasionally learns to follow the reference path and make progress toward the goal. Toward the end of training, high positive rewards become more frequent and the average reward increases, confirming policy improvement. However, several large negative spikes(down to approximately −55) are still present, revealing that the learned policy remains sensitive to certain states and can occasionally fail catastrophically.

The **Double Dueling DQN** shows an initial negative reward plateau (around −10 to −15), followed by an earlier transition to consistent positive rewards compared to DQN. From mid-training onward, high rewards (up to +25) become frequent and stable, while extreme negative drops are largely avoided.

Overall, the Double Dueling DQN converges faster and achieves more stable and higher performance than the DQN.





2.3 Results : Evaluation

Model	Split	Dynamic	MeanReward	StdReward	MeanLength
DQN	train	FALSE	-2.526709	29.052217	388.4
DQN	test	FALSE	17.138081	17.747901	96.88
DQN	train	TRUE	-3.280604	28.051261	331.42
DQN	test	TRUE	7.198174	26.153857	113.58
DoubleDueling DQN	train	FALSE	20.935151	6.436912	93.02
DoubleDueling DQN	test	FALSE	23.377219	6.524525	87.52
DoubleDueling DQN	train	TRUE	18.658048	9.079358	95.5
DoubleDueling DQN	test	TRUE	16.018662	12.27502	77.46

1.Performance level: Double Dueling DQN dominates DQN in all settings. In test/static it reaches 23.38 vs 17.14, and in test/dynamic it reaches 16.02 vs 7.20.
2.Stability (StdReward): DQN has extremely high variance (~26–29), indicating frequent unstable episodes (collisions / large deviations). Double Dueling DQN is much more stable (~6.4–12.3), consistent with improved value estimation.
DQN has extremely high variance (~26–29), indicating frequent unstable episodes. Double Dueling DQN is much more stable (~6.4–12.3).
3.Effect of dynamic obstacles: Dynamics reduce MeanReward for both methods, but the drop is much larger for DQN (test: −9.94) than for Double Dueling (test: −7.36), and DQN remains high-variance. This confirms Double Dueling learns a more robust navigation policy under distribution shift.
4.Episode length:DQN's very large MeanLength in train (331–388 steps) combined with near-zero/negative reward indicates many episodes are prolonged but unproductive. Double Dueling achieves high reward with short episodes (~87–96 steps).

2.4 Discuss

The evaluation shows a clear gap between the two approaches. Double Dueling DQN consistently achieves higher mean reward and much lower variance across all settings, while standard DQN remains unstable. In static scenarios, Double Dueling DQN reaches strong performance on both splits (train/test mean reward ≈ 20.9–23.4 with std ≈ 6.4–6.5) and short episode lengths (~88–93 steps), which indicates an efficient policy that follows the planned path with low lateral error and reaches the goal reliably.

In contrast, the DQN results present high reward variance (std ≈ 26–29) and a strong discrepancy between splits. In the train split, DQN has negative mean reward and very long episodes (mean length ≈ 331–388), suggesting frequent inefficient behavior: the agent often fails to reduce path error or progress sufficiently, accumulating time and distance penalties over long rollouts and occasionally triggering collision penalties, which explains the large standard deviation. The test split under static obstacles is better (mean reward ≈ 17.1), but still shows substantial variance and therefore lower reliability compared to Double Dueling DQN.

Introducing dynamic obstacles degrades performance for both models, as expected, because moving agents create state transitions that are not fully covered by static training/evaluation. The effect is most severe for DQN, whose test mean reward drops from ~17.1 to ~7.2 and remains highly variable. Double Dueling DQN remains robust (test mean reward still positive at ~16.0), but the increase in variance (std rises to ~12.3) indicates that some episodes become significantly harder due to dynamic interactions (e.g., occasional collisions or forced detours). Overall, the results suggest that Double Dueling DQN generalizes better and is more robust, while DQN is sensitive to scenario changes and lacks stability.

Possible improvements: To further improve performance, especially in scenarios with dynamic obstacles, the model should be trained with dynamic obstacles enabled or using a curriculum learning strategy that gradually increases environment complexity. In addition, the DIRECTION_AND_ERROR observation could be extended with local obstacle information to provide better awareness of nearby dynamic agents. Finally, minor reward shaping and hyperparameter tuning (e.g., slower epsilon decay or larger replay buffer) could improve stability and reduce performance variance.